

Projeto de Engenharia de Software 2025/26

(É da responsabilidade do aluno a leitura atenta do enunciado e cumprir os requisitos requeridos, como, por exemplo, no que diz respeito à obrigatoriedade de atualização, a cada tarefa, do ficheiro do Relatório de Entrega)

Na unidade curricular de Engenharia de Software, iremos estender a aplicação [HumanaEthica](#), um sistema aberto de gestão de voluntariado que permite ligar instituições de apoio social com voluntários. A plataforma permite a membros de instituições proporem atividades às quais voluntários se podem candidatar.

O objetivo é reestruturar a aplicação de forma que uma atividade possa ter vários turnos e voluntários concorram aos turnos.

Para facilitar o desenvolvimento, estão disponíveis vídeos sobre a aplicação, sobre algumas das suas funcionalidades, modelos e código. Adicionalmente, todas as funcionalidades podem ser experimentadas usando as interfaces DEMO disponibilizadas.

Estrutura do projeto

O projeto é efetuado em duas partes, com duas entregas. Existirá uma discussão final, em que cada aluno realizará uma tarefa individual sobre o código final da solução fornecida pelo corpo docente. Como resultado da discussão, será verificado se a nota atribuída ao seu projeto é adequada.

Na primeira parte, implementa-se a parte servidor da aplicação (*backend*), enquanto que na segunda parte se implementa a parte cliente (*frontend*).

A primeira parte será desenvolvida sobre o código base fornecido pelo corpo docente. A segunda parte será desenvolvida sobre o código da solução da primeira parte, também fornecido pelo corpo docente. Note que o código base da segunda parte pode conter a implementação de funcionalidades adicionais. Além disso, na resolução da segunda parte, pode ser necessário fazer alterações ao código do lado do *backend* (por exemplo, colocar mais dados nos DTOs ou acrescentar serviços web).

A implementação pré-existente das funcionalidades associadas à entidade Activity no código de base fornecido serve de referência para o desenvolvimento a efetuar durante o projeto.

O projeto é realizado por grupos de 2 alunos.

Entregas

O projeto tem duas entregas:

1. (50%) Parte *backend*. Entrega a **13 de março de 2026 às 17:00**
2. (50%) Parte *frontend*. Entrega a **2 de abril de 2026 às 17:00**

As entregas são efetuadas no GitLab RNL, usando o ramo *sprint-1* para a primeira entrega e o ramo *sprint-2* para a segunda entrega, que são criados a partir do ramo *main*, quando o código estiver terminado.

O ramo não pode ser alterado após a data e hora estipuladas (isto será verificado pelos docentes).

Exemplo de como criar o ramo da 1ª entrega:

```
git checkout main
git checkout -b sprint-1
git push origin sprint-1
```

Repositório de código

O projeto será desenvolvido no GitLab da RNL. Para se registarem, basta fazer login via fénix: <https://gitlab.ml.tecnico.ulisboa.pt>.

O código do vosso grupo estará disponível em <https://gitlab.ml.tecnico.ulisboa.pt/es/es26-CC-XX> (onde CC é o campus e XX é o número do grupo). Se ainda não tinham criado conta no GitLab, pode demorar alguns dias até terem permissões de acesso ao vosso projeto.

NB: devem configurar o cliente de git com o vosso nome e e-mail. Se usarem o GIT na consola, devem correr estes comandos:

```
git config --global user.name "Nome Apelido"
git config --global user.email "your.email@tecnico.ulisboa.pt"
```

O `--global` altera a configuração para todos os repositórios. Alternativamente, podem correr o comando apenas no diretório do projeto sem `--global`.

Para interagir com o repositório sem necessitar de se autenticar, pode-se criar uma chave ssh no computador:

```
ssh-keygen -t ed25519 -C "your.email@tecnico.ulisboa.pt"
```

Quando solicitado, dar o nome `ist_gitlab_rsa` e depois fazer:

```
cat ~/.ssh/ist_gitlab_rsa.pub
```

E colocar o conteúdo no GitLab em "Edit Profile->SSL Keys".

Finalmente, editar/criar `~/.ssh/config`, adicionando:

```
Host gitlab.ml.tecnico.ulisboa.pt
IdentityFile=~/.ssh/ist_gitlab_rsa
```

Para fazer clone do projeto:

```
git clone git@gitlab.ml.tecnico.ulisboa.pt:es/es25-CC-XX.git
```

Instalação / Configuração do Ambiente de Desenvolvimento

Existem duas forma de instalar e configurar o ambiente de desenvolvimento: bare metal e docker. As instruções estão no [README](#) do projeto.

Qualquer IDE pode ser utilizado, mas recomendamos a utilização de VS Code ou IntelliJ IDEA (Ultimate Edition).

O código poderá ser desenvolvido usando plataformas como CoPilot (<https://github.com/features/copilot>) e Antigravity (<https://antigravity.google/>). Contudo, os alunos devem conhecer e rever todo o código gerado. Serão feitas perguntas sobre esse código no exame e na discussão de projeto.

Grupos, Tarefas e Ramos

O projeto é realizado por grupos de 2 alunos.

Filosofia de desenvolvimento **obrigatória** (os grupos que não a seguirem serão penalizados):

- No repositório Git existirá um ramo (*branch*) principal denominado *main*. Este ramo é partilhado pelos dois alunos. O *buildbot* do *main* deve estar sempre verde;
- O trabalho é dividido em tarefas, e **cada tarefa deve ter uma granularidade pequena, tipicamente, deve poder ser**

[Página Inicial](#) 

[Grupos](#)

[Avaliação](#)

[Bibliografia](#)

[Horário](#)

[Métodos de Avaliação](#)

[Objectivos](#)

[Planeamento](#)

[Programa](#)

[Turnos](#)

[Anúncios](#) 

[Sumários](#) 

[Pré-requisitos](#)

[Componente Laboratorial](#)

[Componente de Programação e Computação](#)

[Competências Transversais](#)

[Princípios Éticos](#)

[Notas](#)

[Horários de Dúvidas](#)

[Aulas de Laboratório](#)

[Aulas Teóricas](#)

[Código de Conduta para Utilização do Mattermost](#)

[Projecto](#)

[Primeira Entrega \(backend\)](#)

- Implementada num curto espaço de tempo;
- A cada tarefa é associado um *issue* no *GitLab* e integrado no planeamento (ver Sugestões de Planeamento);
- Cada tarefa é **implementada por um, e apenas um, elemento do grupo** que deve ser *assigned* ao *issue* da tarefa;
- Para a realização de cada tarefa deve ser criado um novo ramo. Após a sua conclusão, deve ser feito um *commit* associado ao *issue* correspondente, incluindo no final da mensagem de *commit* a expressão “Closes #n”, sendo n o número do *issue*.
- O conjunto de ficheiros alterados na tarefa, e que fazem parte do *commit*, deve incluir o ficheiro de relatório de entrega (explicado abaixo), atualizado com a informação sobre a tarefa;
- Após terminar a tarefa, o aluno deve fazer *fetch* do *main* e integrar com o ramo da tarefa, para assegurar que vai conter o código mais recente;
- Após a integração com a versão mais recente do código, o ramo da tarefa deve ser submetido como *Merge Request* (MR) para revisão por um colega do grupo;
- Se, e quando o outro elemento do grupo aprova o MR, este é integrado no ramo principal, *main*;
- Cada elemento do grupo deve ser responsável por um número aproximadamente equivalente de tarefas.
- As mensagens de *commit* devem seguir o padrão de <https://www.conventionalcommits.org/en/v1.0.0>;
- Todos os *commits* devem ser feitos utilizando o nome real e o endereço de email do IST.

Sugestões de Planeamento

1. Ler com atenção o enunciado;
2. Analisar o código fornecido como suporte;
3. Planear:
4.
 1. Definir as histórias para cada uma das funcionalidades;
 2. Decompor as histórias em tarefas de modo a haver um desenvolvimento incremental que permita validação constante;
 3. Cada tarefa deve incluir os seus testes. Todo o código submetido ao *main* através de MR deve estar completamente funcional e testado.

Cada tarefa estará associada a um elemento do grupo e um *commit* a realizar por esse elemento (por exemplo, implementação e teste de um invariante).

Deve haver um Board por sprint (entrega), com 6 colunas:

- *Sprint backlog*
- *Tasks ready*
- *In progress tasks*
- *In review tasks*
- *Tasks done*
- *Accepted stories*

Relatório de Entrega

Em cada entrega deve ser incluído um ficheiro markdown com uma descrição detalhada das alterações realizadas. Esse ficheiro está no diretório *delivery-reports*, [P1.md](#) para a primeira entrega e [P2.md](#) para a segunda entrega.

Por forma a serem avaliados, as seguintes condições devem ser tidas em consideração:

- Se não existir o ficheiro, não são avaliados;
- Apenas o que está reportado no ficheiro será considerado na avaliação;
- Se o ficheiro não estiver no formato fornecido, serão penalizados nos itens que não respeitem o formato;
- Se colocarem informação no ficheiro falsa, por exemplo, uma cobertura que não corresponde à realidade, ou um *commit* de uma tarefa que não existe, serão penalizados;
- Quando terminarem uma tarefa, a atualização do ficheiro relatório de entrega é obrigatória e deve ser incluída no *commit*.

Modelo de Domínio

Atualmente, o sistema possui o seguinte modelo de domínio:

